SWEN 1

Media Ratings Platform (MRP): Dokumentation

Opeyemi Akinrinlola if25b072

---

1. Projektüberblick

Die Media Ratings Platform ist eine Java REST API zur Verwaltung von Benutzern und Medieneinträgen. Die Anwendung nutzt den eingebauten Java HttpServer, PostgreSQL für die dauerhafte Speicherung, JDBC für den Datenbankzugriff und JWT für die Authentifizierung.

Dieses Repository enthält die Zwischenabgabe des Projekts.

GitHub: https://github.com/Yemi-dot-X/media-ratings-platform

2. Technologien

Technologie	Zweck
Java	Backend-Anwendung
Java HttpServer	HTTP-Anfragen und Routing
PostgreSQL 16	Relationale Datenbank
JDBC	Datenbankzugriffe
Docker Compose	Datenbank-Setup
Jackson	JSON-Serialisierung und -Deserialisierung
JWT	Authentifizierung
JUnit	Automatisierte Tests
Postman	API-Tests

3. Architektur

Die Anwendung trennt die Verantwortlichkeiten in Schichten:

- **Model**: Stellt die Anwendungsdaten dar.
- **RouteHandler**: Leitet eingehende HTTP-Anfragen weiter.
- **Controller**: Verarbeitet HTTP-Anfragen und Antworten.
- **Service**: Enthält Geschäftslogik und Validierung.

- **Repository**: Führt Datenbankoperationen aus.

Beispiel für den Ablauf einer Anfrage:

Postman → MediaRouteHandler → MediaController → MediaService → JdbcMediaRepository → PostgreSQL

Technische Begründung:

- **Java HttpServer:** Ich habe mich für den Java HttpServer entschieden, weil ich die Verarbeitung von HTTP-Anfragen selbst verstehen und umsetzen möchte. Ein zusätzliches Framework ist für den aktuellen Projektumfang nicht notwendig.
- **Schichtenarchitektur:** Trennt Handler, Service und Repository, weil jede Schicht eine eigene Verantwortung übernimmt. Dadurch erkenne ich leichter, wo ein Fehler entsteht und wo Änderungen sinnvoll sind.
- **Repository mit JDBC:** Ich verwende JDBC, um SQL-Abfragen selbst zu schreiben und die Kommunikation zwischen Java und PostgreSQL besser zu verstehen. Die Trennung erleichtert außerdem das Testen der Datenbankzugriffe.
- **Parametrisierte Abfragen:** Ich verwende PreparedStatements, damit Benutzereingaben nicht als SQL-Befehle interpretiert werden. Dadurch reduziere ich das Risiko einer SQL-Injection.
- **PostgreSQL**: Ich habe eine relationale Datenbank gewählt, weil Benutzer, Medien und Bewertungen miteinander verbunden sind. Primär- und Fremdschlüssel helfen, diese Beziehungen konsistent zu halten.
- **Docker Compose:** Startet die Datenbank mit einer festgelegten Konfiguration gestartet wird. Dadurch reduziere ich Unterschiede zwischen Entwicklungsumgebungen.
- **JWT:** damit der Server Benutzer anhand eines signierten Tokens authentifizieren kann, ohne klassische serverseitige Sessions zu speichern.
- **Jackson:** JSON-Anfragen in Java-Objekte umzuwandeln und Antworten als JSON auszugeben. Dadurch konzentriere ich mich auf die Anwendungslogik statt auf einen eigenen JSON-Parser.
- **Autorisierung im Service**: Prüft im Service, ob der angemeldete Benutzer berechtigt ist, einen Medieneintrag zu ändern oder zu löschen. Diese Entscheidung gehört zur Geschäftslogik und soll unabhängig vom HTTP-Handler funktionieren.
- **JUnit:** Ich verwende automatisierte Tests, um zu überprüfen, ob meine Methoden unter verschiedenen Bedingungen korrekt funktionieren. Dadurch muss ich mich nicht allein auf manuelle Tests verlassen.

- **Postman:** die API unabhängig von einem Frontend zu testen. Ich überprüfe erfolgreiche Anfragen, ungültige Eingaben und die Authentifizierung mit JWT.

4. Datenbank

Die PostgreSQL-Datenbank läuft in Docker. Das Projekt enthält docker-compose.yml und src/main/resources/schema.sql.

Das Schema definiert die Tabellen users und media mit Primärschlüsseln und der Fremdschlüsselbeziehung zwischen Medieneinträgen und ihren Erstellern.

Das SQL-Initialisierungsskript läuft nur, wenn PostgreSQL ein neues, leeres Datenbank-Volume initialisiert.

Die Verbindungseinstellungen in der Java-Anwendung müssen zur Docker-Compose-Konfiguration passen. Der JWT-Signaturschlüssel ist vom Datenbankpasswort getrennt. Die lokalen Zugangsdaten sind nur für Entwicklung und akademische Tests gedacht. Sie dürfen nicht in Produktion verwendet werden.

5. Authentifizierung

Die API nutzt JWT Bearer Authentication.

1. Benutzer über POST /api/users/register registrieren.
2. POST /api/users/login ausführen.
3. Die Login-Antwort enthält das JWT im Feld token.
4. Postman speichert den Token automatisch.
5. Geschützte Endpunkte verwenden den gespeicherten Token.

6. Implementierte Media-Endpunkte

Methode	Endpunkt	Beschreibung	Authentifizierung
GET	/api/media	Medien auflisten, suchen und filtern	Nein
GET	/api/media/{id}	Medieneintrag abrufen	Nein
POST	/api/media	Medieneintrag erstellen	JWT
PUT	/api/media/{id}	Medieneintrag ändern	JWT
DELETE	/api/media/{id}	Medieneintrag löschen	JWT

Nur der Ersteller eines Medieneintrags darf ihn ändern oder löschen.

Beispiel: Medieneintrag erstellen

POST /api/media

```
{
  "title": "Interstellar",
  "description": "Science Fiction",
  "mediaType": "MOVIE",
  "releaseYear": 2014,
  "ageRestriction": 12,
  "duration": 169,
  "genres": ["ACTION"]
}
```

Eine erfolgreiche Erstellung liefert HTTP 201 Created.

Suche und Filter

GET /api/media?title=Interstellar

GET /api/media?genre=ACTION

GET /api/media?mediaType=MOVIE

GET /api/media?sortBy=releaseYear

7. HTTP-Statuscodes

Die folgenden Antworten wurden mit Postman geprüft:

Status	Beschreibung
200 OK	Erfolgreiches GET oder PUT
201 Created	Medieneintrag erstellt
204 No Content	Medieneintrag gelöscht
400 Bad Request	Ungültige Media-ID
401 Unauthorized	JWT fehlt oder ist ungültig
403 Forbidden	Benutzer besitzt den Medieneintrag nicht
404 Not Found	Medieneintrag existiert nicht
405 Method Not Allowed	HTTP-Methode nicht unterstützt

8. Tests

Das Projekt enthält JUnit-Tests für Repository-Operationen:

- Create
- Read
- Update
- Delete
- Search

Die Media API wurde zusätzlich mit Postman getestet, einschließlich Authentifizierung, Autorisierung, Filterung und Sortierung.

Postman-Tests ausführen:

1. Die mitgelieferte Postman Collection importieren.
2. PostgreSQL und die Java-Anwendung starten.
3. Mit einem Testbenutzer registrieren oder anmelden.
4. Die Media-Anfragen ausführen.
5. HTTP-Statuscodes und JSON-Antworten prüfen.

Die Collection enthält zusätzliche Anfragen für geplante Funktionen. Ihr Vorhandensein bedeutet nicht, dass diese Funktionen umgesetzt sind.

9. Aktueller Projektumfang

Die Zwischenabgabe umfasst Benutzer-Authentifizierung und Medienverwaltung.

Weitere geplante Funktionen sind Bewertungen, Favoriten, Empfehlungen und Leaderboard. Sie sind in dieser Zwischenabgabe nicht als fertig dokumentiert.

10. Sicherheit und Persistenz

- PostgreSQL speichert die Anwendungsdaten dauerhaft.
- Docker Compose stellt die lokale Datenbankumgebung bereit.
- Das Datenbankschema liegt in src/main/resources/schema.sql.
- Die Authentifizierung nutzt JWT Bearer Tokens.
- SQL-Anweisungen verwenden parametrisierte Abfragen als Schutz vor SQL-Injection.

Estimated Time Tracking

Project Component	Estimated Time
Requirements Analysis and Planning	5 hours
Project Setup and Architecture	4 hours
Database Design and PostgreSQL Setup	4 hours
User Registration and JWT Authentication	8 hours
Media Management (CRUD)	8 hours
JDBC Repository Implementation	4 hours
Error Handling and Validation	4 hours
Unit Testing with JUnit	2 hours
API Testing with Postman	4 hours
Documentation and Final Review	3 hours